

HW 6

Licza Lobo

11/19/2024

What is the difference between gradient descent and *stochastic* gradient descent as discussed in class? (*You need not give full details of each algorithm. Instead you can describe what each does and provide the update step for each. Make sure that in providing the update step for each algorithm you emphasize what is different and why.*)

Gradient Descent computes a vector of partial derivatives, representing the direction of steepest descent. In Federated Learning, this is used to minimize the loss function and update model parameters. The update step using Gradient Descent is: $\theta_{i+1} = \theta_i - \alpha \nabla f(\theta_i, X, Y)$ where X and Y represent the entire dataset. The gradient is computed using the whole dataset, which can result in the function getting stuck in a local minimum rather than the global minimum. To avoid this, we use the Stochastic Gradient Descent (SGD). SGD computes a vector of partial derivatives at each step with a subset of the data rather than the whole dataset. This is represented in the update step as: $\theta_{i+1} = \theta_i - \alpha \nabla f(\theta_i, X'_i, Y'_i)$. This allows SGD to help the function converge to a global minimum.

Consider the **FedAve** algorithm. In its most compact form we said the update step is $\omega_{t+1} = \omega_t - \eta \sum_{k=1}^K \frac{n_k}{n} \nabla F_k(\omega_t)$. However, we also emphasized a more intuitive, yet equivalent, formulation given by $\omega_{t+1}^k = \omega_t - \eta \nabla F_k(\omega_t); w_{t+1} = \sum_{k=1}^K \frac{n_k}{n} w_{t+1}^k$.

Prove that these two formulations are equivalent.

(*Hint: show that if you place ω_{t+1}^k from the first equation (of the second formulation) into the second equation (of the second formulation), this second formulation will reduce to exactly the first formulation.*)

$$w_{t+1} = \sum_{k=1}^K \frac{n_k}{n} w_{t+1}^k = \sum_{k=1}^K \frac{n_k}{n} (\omega_t - \eta \nabla F_k(\omega_t)) = \sum_{k=1}^K \frac{n_k}{n} (\omega_t) - \left(\frac{n_k}{n}\right) (\eta \nabla F_k(\omega_t)) = (\omega_t) \left(\sum_{k=1}^K \frac{n_k}{n}\right) - \eta \sum_{k=1}^K \left(\frac{n_k}{n}\right) \nabla F_k(\omega_t)$$

$$\text{Note, } \sum_{k=1}^K \frac{n_k}{n} = 1, \text{ so } (\omega_t) \left(\sum_{k=1}^K \frac{n_k}{n}\right) - \eta \sum_{k=1}^K \left(\frac{n_k}{n}\right) \nabla F_k(\omega_t) = (\omega_t) - \eta \sum_{k=1}^K \left(\frac{n_k}{n}\right) \nabla F_k(\omega_t) = \omega_{t+1}$$

Now give a brief explanation as to why the second formulation is more intuitive. That is, you should be able to explain broadly what this update is doing.

The second formulation is more intuitive because it separates the update process into two steps. First, $\omega_{t+1}^k = \omega_t - \eta \nabla F_k(\omega_t)$ shows how each client k is updated with its local data. Then, $w_{t+1} = \sum_{k=1}^K \frac{n_k}{n} w_{t+1}^k$ shows how these individual updates are averaged according to their weight, $\left(\frac{n_k}{n}\right)$, to give the global update. Thus, the second formulation clearly shows how the FedAve algorithm works as a weighted average of the local models, where the weight for each model is proportional to the amount of data at each client.

Prove that randomized-response differential privacy is ϵ -differentially private.

An algorithm is ϵ -differentially private when for every dataset differing by exactly one element, the ratio of the probabilities for any output is less than e^ϵ . Consider the generalized form of randomized-response differential privacy, $\hat{P} = \frac{\hat{\pi} - (1-\theta)\theta}{\theta}$. This can be rewritten as $\hat{\pi} = \theta(\hat{P}) + (1-\theta)\theta$. When $\theta = 1/2$, we see $\hat{\pi} = \frac{\hat{P}}{2} + \frac{1}{4}$. When considering the case where the ground truth is “yes,” we see that the probability of an output being “yes” is $\frac{1}{2} + \frac{1}{4} = \frac{3}{4}$. When considering the case where the truth is “no,” we see that the probability of an output being “yes” is $\frac{0}{2} + \frac{1}{4} = \frac{1}{4}$. Both ratios are then equal to $1/3$ and 3 . These can be set equal to e^ϵ to see $\ln(1/3) = \epsilon$ and $\ln(3) = \epsilon$. Since, $\ln(3)$ results in a higher bound, randomized-response differential privacy is $\ln(3)$ -differential privacy ($\epsilon = \ln(3)$).

Define the harm principle. Then, discuss whether the harm principle is *currently* applicable to machine learning models. (*Hint: recall our discussions in the moral philosophy primer as to what grounds agency. You should in effect be arguing whether ML models have achieved agency enough to limit the autonomy of the users of said algorithms.*)

The harm principle states that personal autonomy is restricted when exercising that autonomy results in harm to a moral agent. This principle protects the privacy of users by limiting the autonomy of developers, ensuring that they cannot act in ways that could harm others, such as spreading private data to make a profit. This principle is not currently applicable to machine learning models because machine learning models do not have moral agency. They do not possess the capacity to understand or intend harm like humans can. While machine learning models can cause harm through biased decisions or privacy violations, the harm is due to the design or training data, not the model’s own intentional actions. Therefore, the responsibility for harm should fall onto the developers and the choices they make in creating their models.